pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Files and Folders

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of today you can

Find your way around

See which folder Python is in and what it holds.

```
os.listdir('.')
```

Make and rename folders

A place for each day's file to go.

```
os.mkdir('holdings')
```

Write a holdings file

A header and four bonds, saved as a CSV.

```
open(file_path, 'w', encoding='utf-8')
```

Read it back and total it

Line by line, turning text into numbers.

```
lines = file.readlines()
```

A holdings file, every morning

Ticker	Coupon (%)	Maturity	Price	Face held
QVNE	5.25	2033	98.50	1,500,000
DNMR	6.125	2034	103.25	500,000
PLWK	4.75	2031	95.00	750,000
HLVF	7.50	2032	101.00	250,000

The same four bonds as yesterday. Every issuer is invented.

Two jobs, every morning

File it

Keep each day's file in the right folder.

Open it

Read what is inside and add it up.

Today's route

Look around → Make folders → Write the file → Read it back → Clean up

A module is a toolbox

Ask for it once, with import

```
import os
print(os.name)
nt
```

`nt`

Windows

`posix`

macOS or Linux, and Google Colab

What we take out of it today

Tool	What it does
`os.getcwd()`	Which folder am I in?
`os.listdir()`	What is in this folder?
`os.mkdir()`	Make a folder
`os.rename()`	Rename a folder or a file
`os.remove()`	Delete a file
`os.rmdir()`	Delete an empty folder

Everything in the module is reached with os. and a name.

Python is always sitting in one folder

A path lists every folder, from the top of the drive down

`C:` \ `Users` \ `you` \ `python-fixed-income` \ `course01_python_foundations` \ `week1` \ **`day2`**

An example only. Your path will be different.

the working directory

Where am I?

```
os.getcwd()
```

Just the last folder

```
print(os.path.basename(os.getcwd()))
day2
```

A file name with no folder in front is looked up in the working directory.

A folder's contents come back as a list

What is in the working directory?

```
os.listdir('.')
['02_files_and_folders.ipynb']
```

`'.'` Shorthand for the current working directory.

`[ ]` The answer is a list of names. Everything from yesterday works on it.

Count it

```
print(len(os.listdir('.')))
```

Walk through it

```
for name in os.listdir('.'):
    print(name)
```

Filter it

```
notebooks = [name
    for name in os.listdir('.')
    if name.endswith('.ipynb')]
```

Making and renaming folders

```
os.mkdir('holdings')
```

→ **Quiet** — Nothing is printed. So check that it worked.

```
print(os.path.exists('holdings'))
```

→ **True** — The folder is there.

```
os.mkdir('holdings')
```

→ **FileExistsError** — Run it twice and Python refuses. Nothing inside is touched.

```
os.rename('holdings', 'desk_holdings')
```

→ **Renamed** — The old name first, then the new name.

The backslash trap

Typed by hand

```
print('desk_holdings\new')
desk_holdings
ew
```

Inside a string, \n means: start a new line.

Built with os.path.join

```
day_folder = os.path.join(
    'desk_holdings', '2026-10-02')
print(day_folder)
desk_holdings\2026-10-02
```

You never type the separator yourself.

Where the code runs	Separator	What join builds
Windows	\	desk_holdings\2026-10-02
macOS, Linux, Colab	/	desk_holdings/2026-10-02

Three ways to open a file

`'r'` Read

Look at what is there. Nothing in the file changes.

`'w'` Write

Start from scratch. If the file already exists, it is wiped.

`'a'` Append

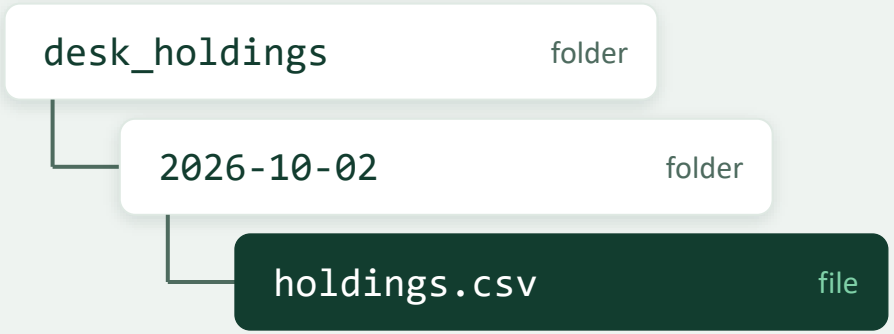
Add to the end. Everything already there is kept.

The safer way to open a file: with

```
with open(file_path, 'a', encoding='utf-8') as file:
    file.write('DNMR,6.125,2034,103.25,500000\n')
```

Python closes the file when the indented block ends.

Where the file lives



Everything read from a file is text

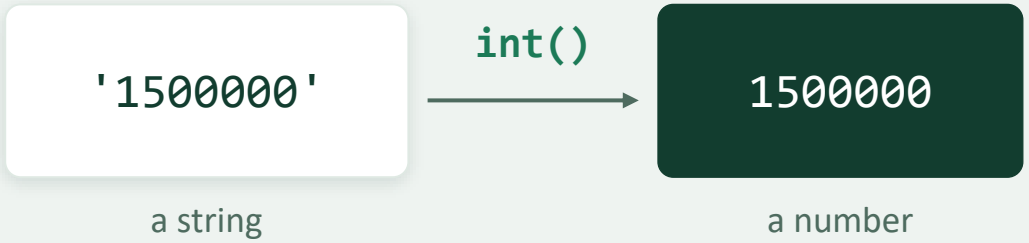
The file, as Python wrote it

```
Ticker,Coupon,Maturity,Price,Face
QVNE,5.25,2033,98.5,1500000
DNMR,6.125,2034,103.25,500000
PLWK,4.75,2031,95.0,750000
HLVF,7.5,2032,101.0,250000
```

Split at the commas: five rows, five columns

Ticker	Coupon	Maturity	Price	Face
QVNE	5.25	2033	98.5	1500000
DNMR	6.125	2034	103.25	500000
PLWK	4.75	2031	95.0	750000
HLVF	7.5	2032	101.0	250000

A value from a file arrives as a string



Convert before any arithmetic

```
total_face = total_face + int(values[4])
```

Forget int() and Python stops with a TypeError.

The csv module, and pathlib

split(',') cuts a quoted value in two. csv.reader does not.

```
import csv
line = 'QVNE,"Quorvane Energy, Inc.",1500000'
print(line.split(','))
['QVNE', '"Quorvane Energy', ' Inc."', '1500000']
for values in csv.reader([line]):
    print(values)
['QVNE', 'Quorvane Energy, Inc.', '1500000']
```

csv.DictReader reads each value by its column name

```
total_face = 0
with open(file_path, 'r', encoding='utf-8',
          newline='') as file:
    for row in csv.DictReader(file):
        total_face = total_face + int(row['Face'])
The total face held is 3,000,000
```

pathlib: a path carries its own answers, reached with a dot. Both styles are in everyday use.

Job	With os	With pathlib
Join the parts of a path	`os.path.join('desk_holdings', '2026-10-02')`	`Path('desk_holdings') / '2026-10-02'`
Check that it exists	`os.path.exists(path)`	`path.exists()`
Take the last part	`os.path.basename(path)`	`path.name`
Find the files that match	`os.listdir() with endswith()`	`path.glob('*.csv')`

Cleaning up, from the inside out

Wrong date on the folder? Rename it. The file moves with it.

```
os.rename(day_folder, new_day_folder)
```

→ `2026-10-02` → **`2026-10-03`**

Then delete, in this order

1 The file

```
os.remove(os.path.join(
    new_day_folder,
    'holdings.csv'))
```

There is no recycle bin.

2 The inner folder

```
os.rmdir(new_day_folder)
```

Works only if the folder is empty.

3 The outer folder

```
os.rmdir('desk_holdings')
```

Back to how we found it.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- See which folder Python is working in and what it holds
- Make, rename, and remove folders, and build paths that work anywhere
- Write a holdings file and add bonds to it
- Read a file line by line, or with the csv module, and total a column
- Turn text from a file into numbers before you calculate

Tomorrow Debugging: finding and fixing the mistakes in code